From the logs in figure 2.6, you'll notice that the startup phase is composed of two main steps:

- Initialization and running of the embedded Tomcat server (which, by default, listens to port 8080 over HTTP)
- Initialization and running of the Spring application context

At this point, you can finally verify whether your HTTP endpoint is working as intended. Open a browser window, navigate to http://localhost:8080/, and be ready to be welcomed to the Polar Bookshop's book catalog.

```
Welcome to the book catalog!
```

The development part of the Polar Bookshop application is done: you have a Catalog Service application welcoming users to the book catalog. Remember to terminate the `bootRun` process (Ctrl-C) to stop the application execution before moving on.

The next step is deploying the application to the cloud. To make it portable on any cloud infrastructure, you should containerize it first. Enter Docker.

## *2.3 Containerizing applications with Docker*

The Catalog Service application is working. Before deploying it to the cloud, however, you should containerize it. Why? Containers provide isolation from the surrounding environment, and they're equipped with all the dependencies required by the application to run.

In our case, most of the dependencies are managed by Gradle and are packaged together with the application (JAR artifact). But the Java runtime is not included. Without a container, you would have to install the Java